
MICROCONTROLLER · PROJECT · MINI CATAPULT

CPE-213 PROJECT

Mini Catapult

A mini catapult for launching small things!

Group Name

มันบด

MEMBERS

ภัทร การะเกด 2411311828

คริสเตียน โดมินิค คาลิก 2411312057

จิรายุ สิมานนท์ 2411380013

Project *Overview*

WHAT IT IS

An automated catapult system designed for precise, repeatable projectile launches. Users set a target launch angle using a rotary encoder and press to start automated winch winding. Once IMU sensor feedback verifies exact angle alignment and the RGB LED signals ready state, pressing the launch button actuates a servo dog clutch to release the arm.

PURPOSE

To build a fun, reliable launching system that eliminates human error by using digital controls for winding, alignment, and quick release.

SCOPE

In Scope:

- Dialing an angle on the screen with a knob.
- Motor automatically pulling the arm down.
- Sensor reading the exact arm angle.
- RGB LED showing real-time system state.
- Button triggering the quick release.

Out of Scope:

- Auto-reloading projectiles into the arm.
- Bluetooth or phone app control.
- Automatically calculating launch distances.

OUTCOME

A reliable launch cycle where the user dials in an angle, the winch automatically winds and aligns the arm using IMU feedback, a dynamic RGB LED changes color to reflect each system state, and a button press cleanly triggers the launch release.

Hardware & Components

Controller & UI

- OLED Display (SSD1306 0.96" I2C)
- ESP32 DOIT DEVKIT V1
- Rotary Encoder (KY-040)
- RGB LED

Sensors & Actuators

- MPU6050 (Gyro & Accelerometer)
- DRV8833 Motor Driver
- N20 Gear Motor
- Servo Motor (MG90S)

KEY SPECIFICATIONS

VOLTAGE

5V / 3.3V

CLOCK

240 MHz

I/O PINS

13 GPIO Pins

Hardware & Components

Mechanical & Structural Parts

- 3D Printed Parts (PLA / PETG)
- Ball Bearings (F623ZZ / 623ZZ / MF62ZZ)
- Steel Shafts (3mm & 5mm)
- Heat-set Brass Inserts (M2 / M3)
- Nylon Fishing Line

Power & Wiring

- Power Supply / Battery (5V/12V)
- Jumper Wires
- Breadboard

KEY SPECIFICATIONS

VOLTAGE

5V / 3.3V

CLOCK

240 MHz

I/O PINS

13 GPIO Pins

Requirements & *Specifications*

FUNCTIONAL

- **Target Selection:** Dial in launch angles (0° to 90°) using a rotary encoder with live OLED feedback.
- **Angle Feedback:** Real-time arm angle tracking via MPU6050 IMU using a complementary filter.
- **Automated Winding:** Closed-loop winch motor control (DRV8833 PWM) to pull the arm to the target angle.
- **Precision Alignment:** Automated fine-tuning pulse sequence (settling state) to eliminate overshoots.
- **Quick Release & Feedback:** MG90S servo dog clutch release on button launch, paired with a state-driven RGB LED indicator.

NON-FUNCTIONAL

- **Performance / Precision:** Arm angle target accuracy maintained within $\pm 0.3^\circ$ tolerance.
- **Safety & Fault Handling:** Built-in escape hatch limiting settle retries to 10 to prevent infinite motor stalls.
- **Power Efficiency:** Detaches servo power when engaged to prevent idle current draw and motor heating.
- **Responsiveness:** Debounced button reads (500 ms) and real-time IMU filter sampling (≈ 20 Hz refresh rate).

KEY SPECIFICATIONS

VOLTAGE

5V / 3.3V

CLOCK

240 MHz

I/O PINS

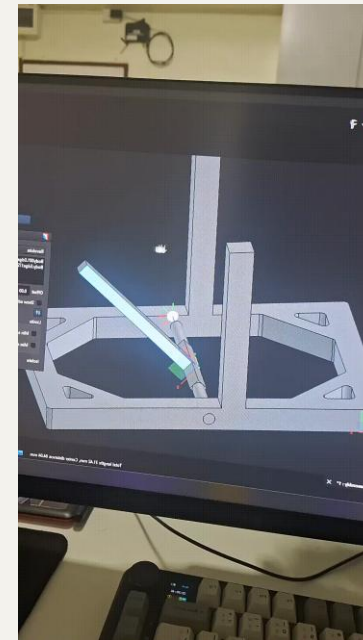
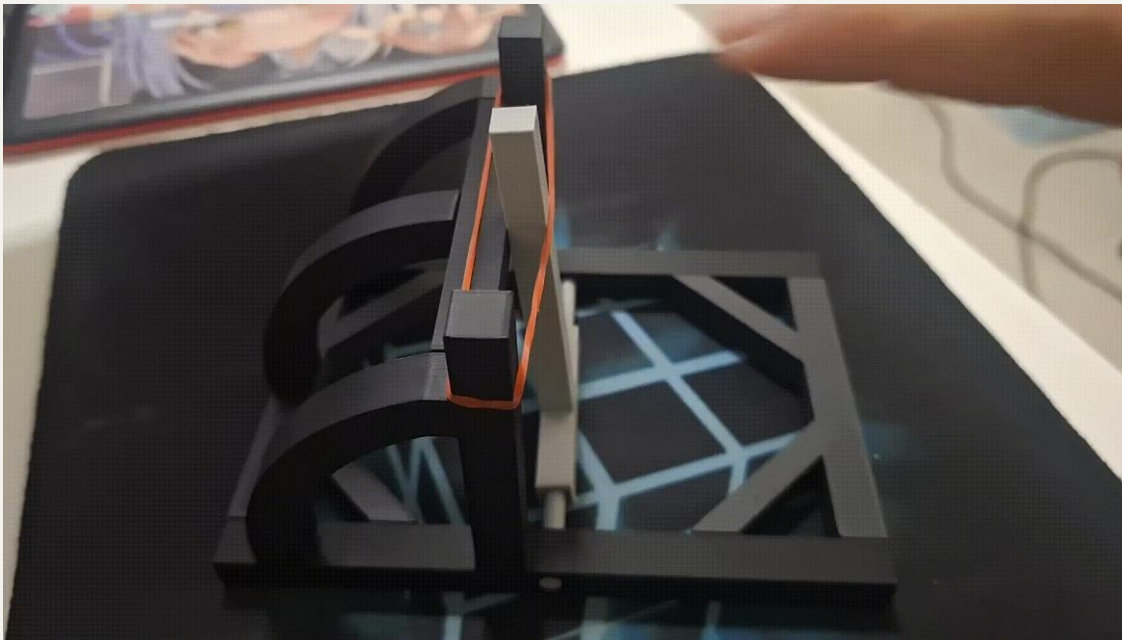
13 GPIO Pins

Project *Inspiration*



Project *Progression*

First Design: Simple looking catapult all made of plastic



Project *Progression*

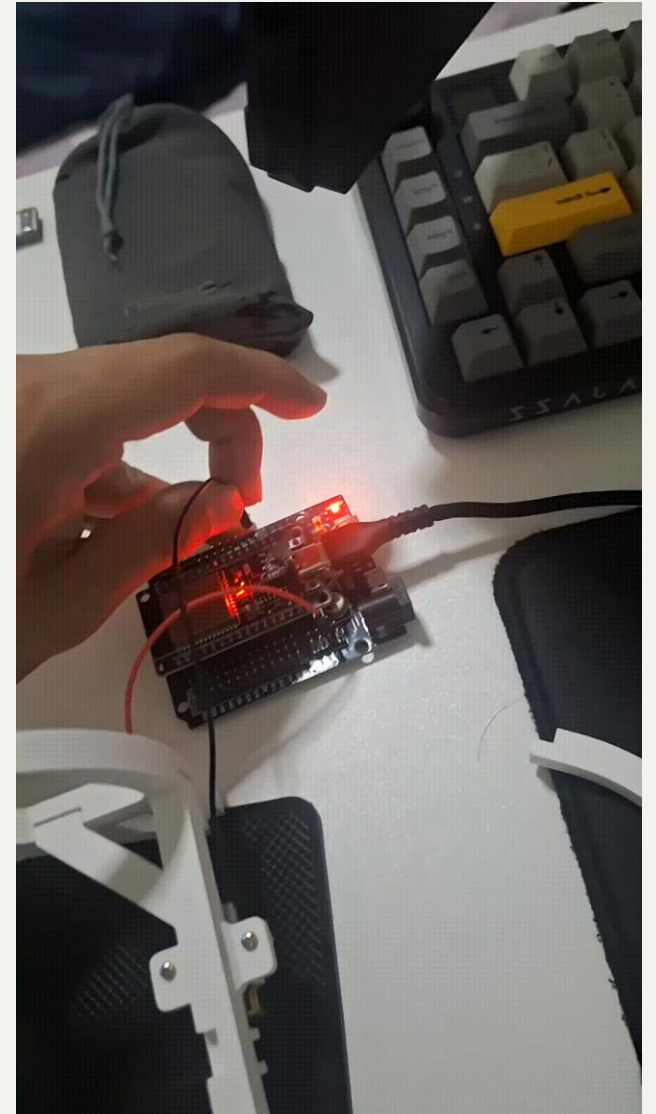
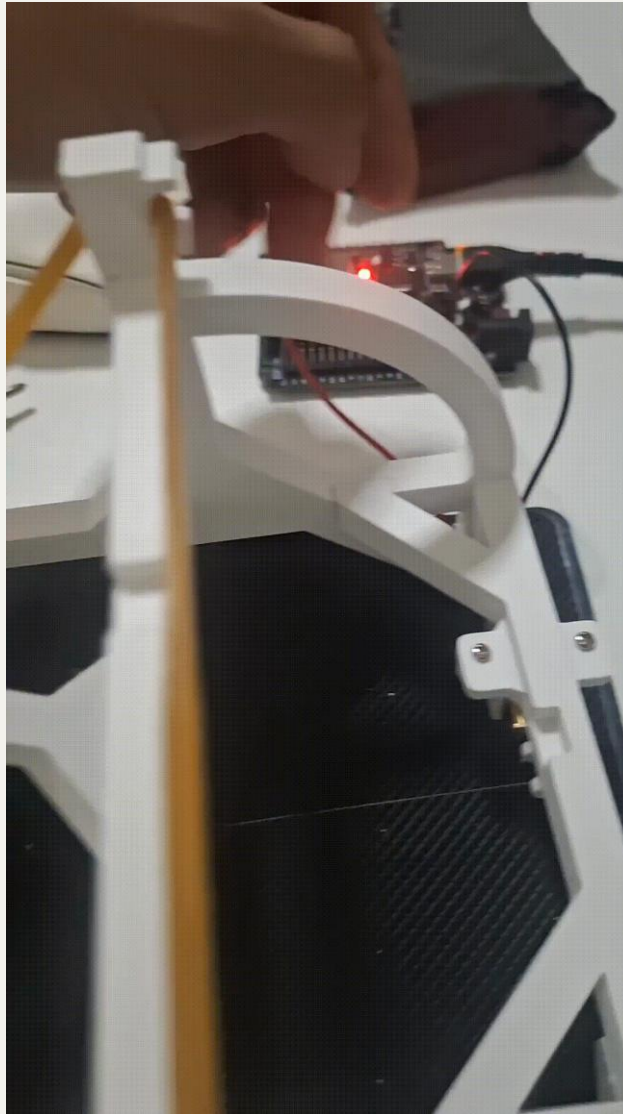
Latch Design:

Latch system to wind and release the catapult arm. but this system is inefficient, and the user must manually reload the arm after every launch.



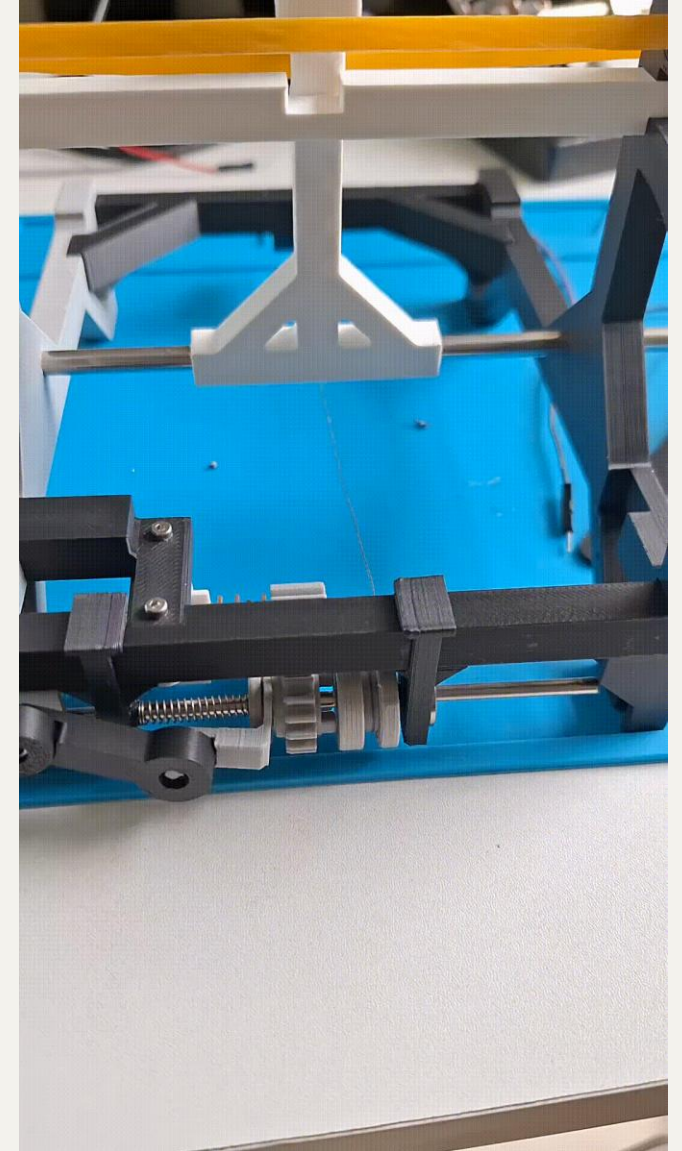
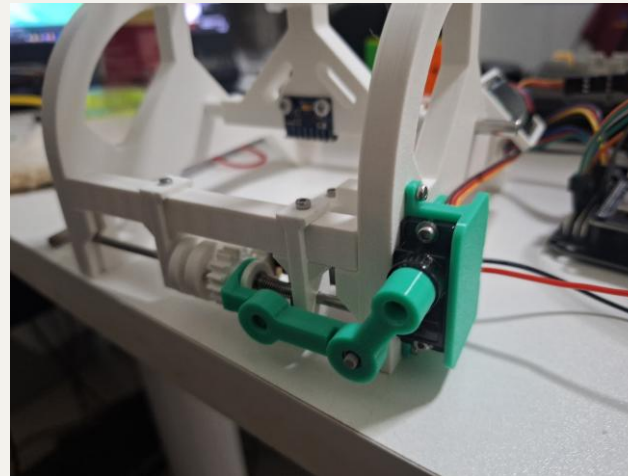
Project *Progression*

Motor for winding:
N20 motor to wind the arm



Project *Progression*

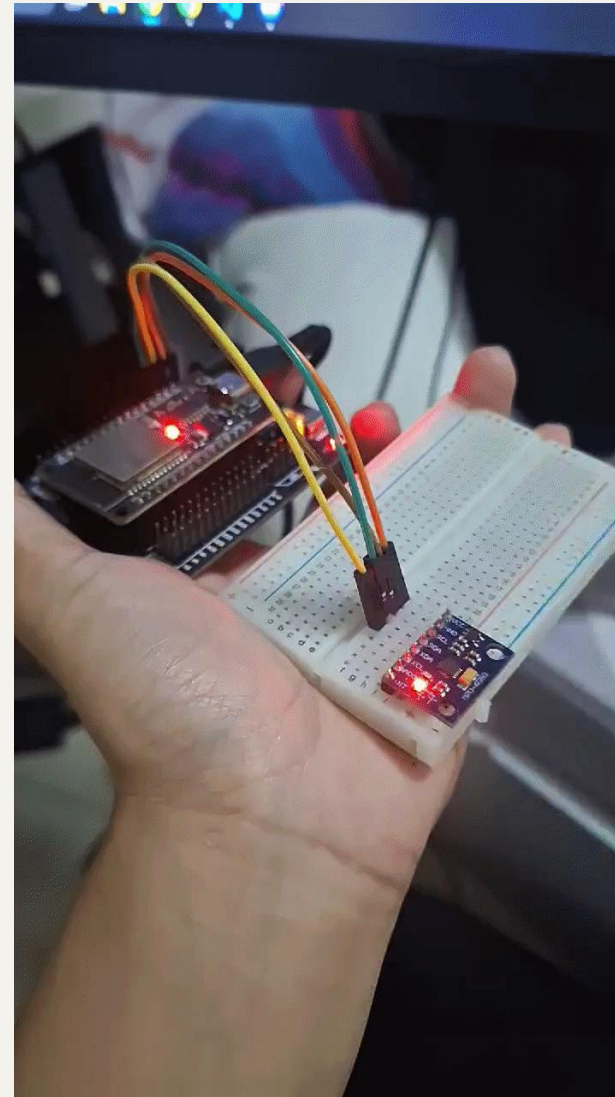
Changed Latch system to Dog Clutch System:
Instead of using the latch system, the Dog Clutch system can both wind and release the arm by just disengaging the dog clutch with the Servo motor.



Project *Progression*

Tracking Angle:

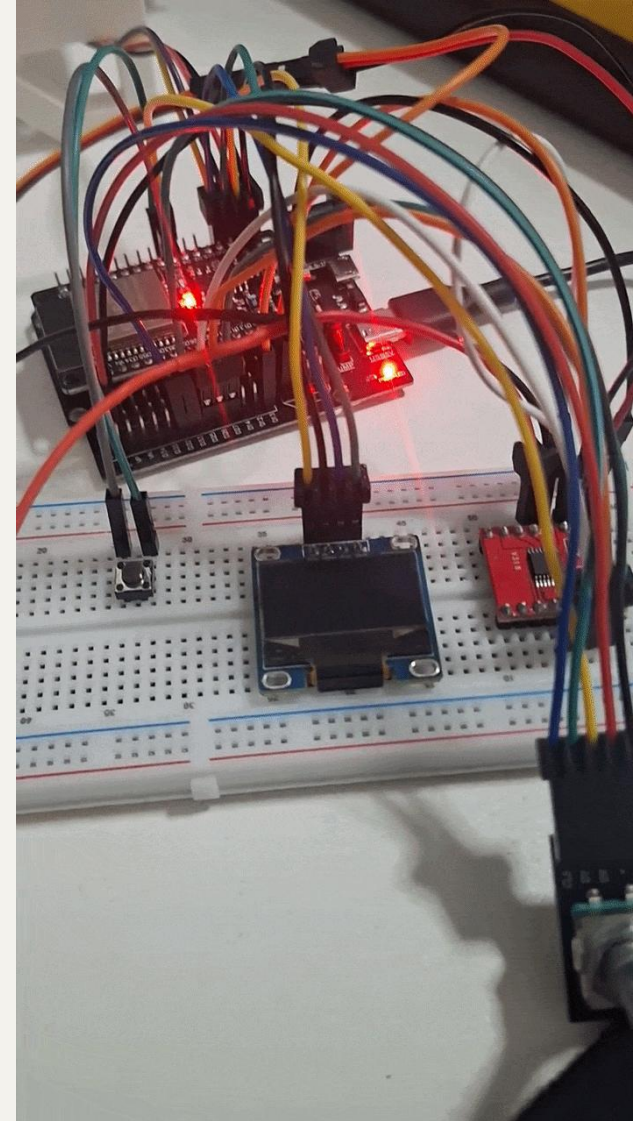
Tracking the angle of the arm using MPU6050



Project *Progression*

Selecting Angle with rotary encoder:

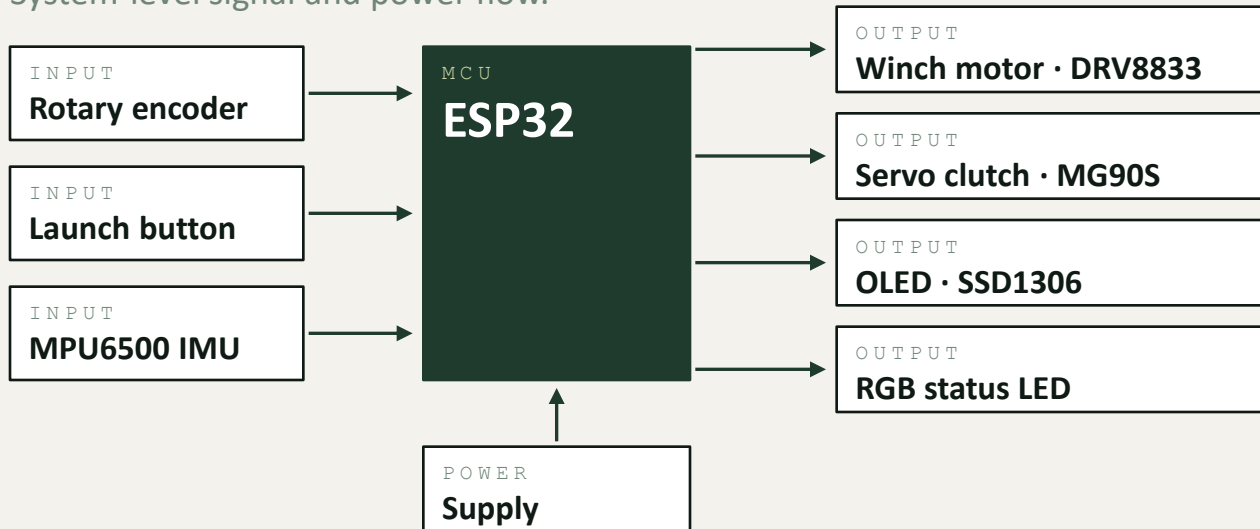
Selecting angles from 0-90 degrees with rotary encoder and confirming by pressing the KY-040's built-in switch.



Architectural Design & *Hardware Selection*

BLOCK DIAGRAM

System-level signal and power flow.



COMPONENTS

The four key parts.



COMPONENT 01

ESP32

Runs the state machine, PWM and I2C.



COMPONENT 02

MPU6050 IMU

Measures arm angle (gyro + accelerometer).



COMPONENT 03

DRV8833 + winch motor

Winds the arm down with PWM speed control.



COMPONENT 04

MG90S servo (dog clutch)

Engages / releases the winch to launch.

Detailed *Design*

STATE MACHINE



EACH PASS OF `loop()`

- 1 update the arm angle
- 2 read the buttons
- 3 run the current state
- 4 refresh OLED + RGB LED

STATE TABLE

SELECT · RAINBOW LED

The encoder sets the target angle (0–90°). Pressing the dial starts winding.

WINDING · RED LED

The winch pulls the arm down, slowing in steps as it nears the target.

SETTLING · YELLOW / ORANGE LED

Waits 1.5 s, checks the error, nudges the motor 45 ms. Up to 10 tries.

READY · GREEN LED

Arm is at the angle. The launch button releases the clutch.

Pin & Hardware *Setup*

```
FILE    main.cpp    ·    CONSTANTS + setup()

// RGB LED
const int RED_PIN = 12, GREEN_PIN = 14, BLUE_PIN = 27;
// Encoder (angle dial) and buttons
const int ENC_CLK = 32, ENC_DT = 33;
const int ENC_SW = 25;          // confirm target angle
const int LAUNCH_BTN = 16;
// Servo clutch
const int SERVO_PIN = 4;
// DRV8833 winch motor
const int AIN1 = 18, AIN2 = 19, SLP = 5;
// Angle limits
const int ANGLE_MIN = 0, ANGLE_MAX = 90;

// I2C bus: MPU6500 + OLED
Wire.begin(21, 22);           // SDA, SCL
```

WHAT THIS BLOCK DOES

All pins and limits are declared as constants at the top of main.cpp, so wiring lives in one place and a pin change is a one-line edit.

KEY POINTS

- 13 GPIO pins used: RGB, encoder, button, servo, motor driver, I2C.
- The MPU6500 and the OLED share one I2C bus (pins 21 and 22).
- Encoder counts 0–180 map to 0–90° (divided by 2).

Servo *Clutch*

```
FILE    main.cpp    ·    engageClutch() / disengageClutch()
```

```
const int ANGLE_ENGAGED    = 0;
const int ANGLE_DISENGAGED = 40;
const int SERVO_MOVE_TIME_MS = 400;

void engageClutch() {
    servoClutch.attach(SERVO_PIN);
    servoClutch.write(ANGLE_ENGAGED);
    delay(SERVO_MOVE_TIME_MS);
    servoClutch.detach();    // spring holds it
}

void disengageClutch() {
    servoClutch.attach(SERVO_PIN);
    // home first: removes drift while detached
    servoClutch.write(ANGLE_ENGAGED);
    delay(SERVO_MOVE_TIME_MS);
    servoClutch.write(ANGLE_DISENGAGED);
    delay(SERVO_MOVE_TIME_MS);
    // stay attached: holds against the spring
}
```

WHAT THIS BLOCK DOES

A servo moves the dog clutch that links the winch to the arm. Engaged lets the winch hold and wind the arm; disengaged releases it to launch.

KEY POINTS

- Engage: write 0°, wait 400 ms, then detach (a spring holds it).
- Disengage: home to engaged first to remove drift, then go to 40°.
- It stays attached after release to hold against the spring.

Winch *Motor*

```
FILE    main.cpp    ·    DRV8833 PWM control
```

```
const int FREQUENCY = 20000;    // 20 kHz PWM
const int RESOLUTION = 8;        // 0-255

void motorForward(int speed) { // winds arm down
    ledcWrite(AIN1_CHANNEL, speed);
    ledcWrite(AIN2_CHANNEL, 0);
}

void motorReverse(int speed) {
    ledcWrite(AIN1_CHANNEL, 0);
    ledcWrite(AIN2_CHANNEL, speed);
}

void motorStop() {                // brake
    ledcWrite(AIN1_CHANNEL, 255);
    ledcWrite(AIN2_CHANNEL, 255);
}

// in setup()
ledcSetup(AIN1_CHANNEL, FREQUENCY, RESOLUTION);
ledcSetup(AIN2_CHANNEL, FREQUENCY, RESOLUTION);
digitalWrite(SLP, HIGH);          // enable driver
```

WHAT THIS BLOCK DOES

The DRV8833 driver spins the winch motor in either direction. Two PWM channels set the speed, and driving both at 255 brakes the motor.

KEY POINTS

- motorForward winds the arm down; motorReverse backs it off.
- 20 kHz, 8-bit PWM (0–255) is above the audible range.
- The SLP pin must be HIGH to enable the driver.

Angle *Sensing*

```
FILE    main.cpp    ·    updateFilteredAngle()

float ax = rawAX / ACCEL_SCALE;    // in g
float ay = rawAY / ACCEL_SCALE;
float az = rawAZ / ACCEL_SCALE;
float gyroY = (rawGY - gyroYOffset) / GYRO_SCALE;

// angle from gravity (accelerometer)
float accelAngleDeg = atan2(ax, az) * 180.0f / PI;

// angle from rotation speed (gyro)
float dt = (nowUs - lastTimeUs) / 1000000.0f;
float gyroAngleDeg = filteredAngleDeg + gyroY * dt;

// trust the accelerometer only when |a| is about 1 g
float mag = sqrt(ax*ax + ay*ay + az*az);
bool trust = (mag > 0.9f && mag < 1.1f);

if (trust)
    filteredAngleDeg = 0.90f * gyroAngleDeg + 0.10f * accelAngleDeg;
else
    filteredAngleDeg = gyroAngleDeg;
```

WHAT THIS BLOCK DOES

A complementary filter blends the gyro (smooth but drifts) with the accelerometer (stable but noisy) to get one reliable arm-angle estimate.

KEY POINTS

- 90% gyro + 10% accelerometer (FILTER_ALPHA = 0.90).
- The gyro offset is calibrated at startup (500 samples).
- The accelerometer is ignored when its magnitude leaves 0.9–1.1 g.

State Machine: *Select & Winding*

```
FILE    main.cpp    ·    loop()    ·    SELECT, WINDING
```

```
enum CatapultState { SELECT, WINDING, SETTLING, READY };

case SELECT:
    targetAngle = clampAngle(encoder.getCount() / 2);
    if (confirmPressed) {
        engageClutch();
        motorForward(255);
        state = WINDING;
    }
    break;

case WINDING: {
    float progress = filteredAngleDeg - targetAngle;
    if (fabs(progress) <= 3)        enterSettling();
    else if (progress <= 30 && progress >= 15)
        motorForward(230);
    else if (progress <= 14 && progress >= 4)
        motorForward(200);
    else if (progress < 0)        enterSettling();
    break;
}
```

WHAT THIS BLOCK DOES

loop() always updates the angle first, then runs the current state. SELECT reads the dial; WINDING runs the motor until the arm is near the target.

KEY POINTS

- The encoder count is clamped to 0–90° while selecting.
- Winding slows in steps (255, 230, 200) as the arm nears the target.
- Within 3° of the target, or past it, it hands off to SETTLING.

Settling & Alignment

```
FILE    main.cpp    ·    loop()    ·    SETTLING

case SETTLING:
    if (settleWaiting) {        // let vibration die down
        if (millis() - settleWaitStart >= SETTLE_DELAY_MS)
            settleWaiting = false;
        break;
    }
    float error = filteredAngleDeg - targetAngle;

    if (fabs(error) <= ANGLE_TOLERANCE_DEG) state = READY;
    else if (settleRetries >= MAX_SETTLE_RETRIES) state = READY;
    else {
        settleRetries++;
        if (error > 0) motorForward(255);    // not low enough
        else          motorReverse(255);    // overshoot
        delay(NUDGE_DURATION_MS);           // 45 ms pulse
        motorStop();
        settleWaiting = true;
        settleWaitStart = millis();
    }
    break;
```

WHAT THIS BLOCK DOES

The motor stops and the arm's vibration dies down, then the angle is rechecked. If it is off by more than 0.3° , a short nudge corrects it.

KEY POINTS

- A 1.5 s wait after each stop keeps the sensor reading trustworthy.
- A 45 ms nudge goes forward or in reverse depending on the error.
- After 10 retries it moves on to READY, where the launch button fires.

Testing & Results

TOTAL CASES

— 3

PASSED

— 3

FAILED

— 0.5

TEST STATUS LEGEND

- Pass
- Partial
- Fail

TEST CASES

CASE 01

Select a target angle with the dial (0–90°).

EXPECTED

Stays within 0–90°

OBSERVED

Works as intended 👍

● Status

CASE 02

Wind and settle at a chosen angle (e.g. 45°).

EXPECTED

Within ±0.3° of the target

OBSERVED

Works as intended 👍

● Status

CASE 03

Press the launch button at 0°

EXPECTED

Servo disengages Dog clutch and launches

OBSERVED

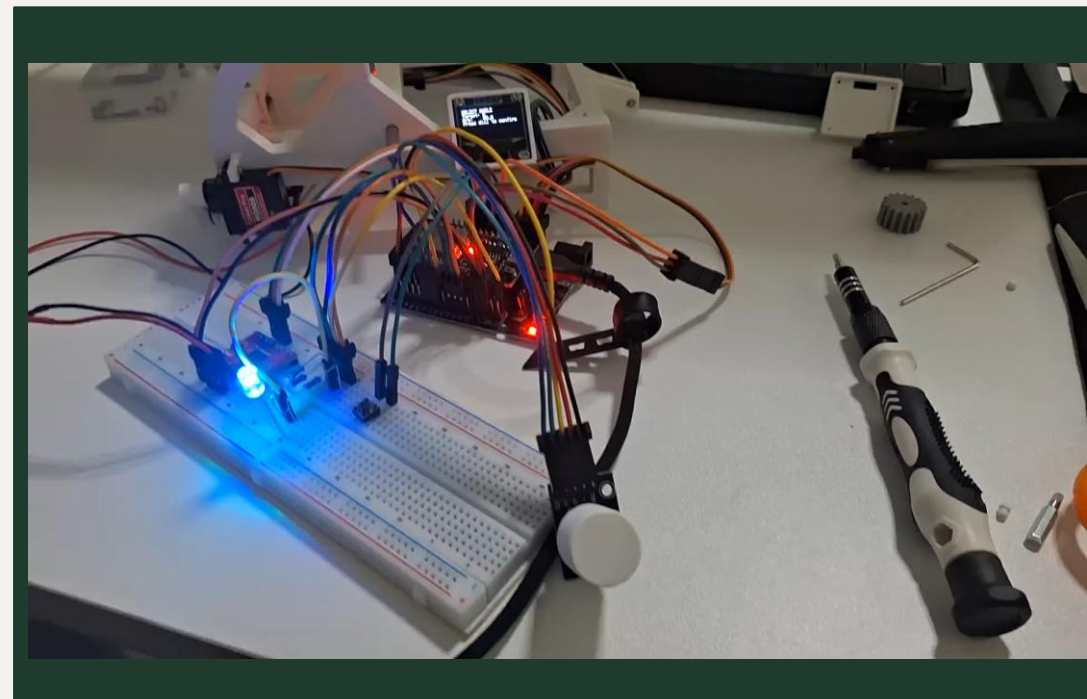
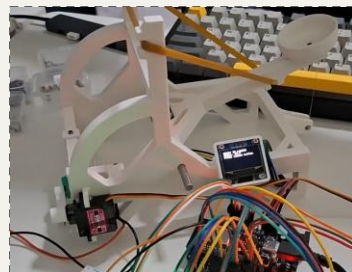
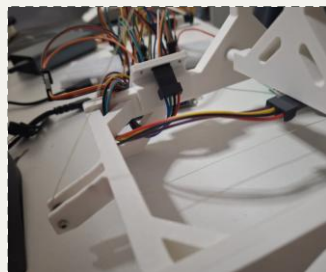
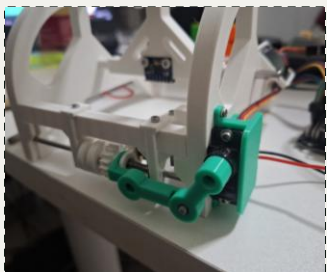
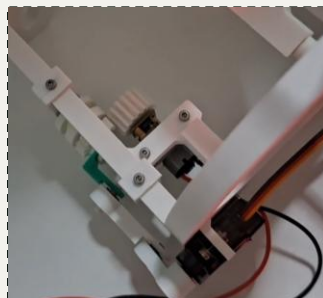
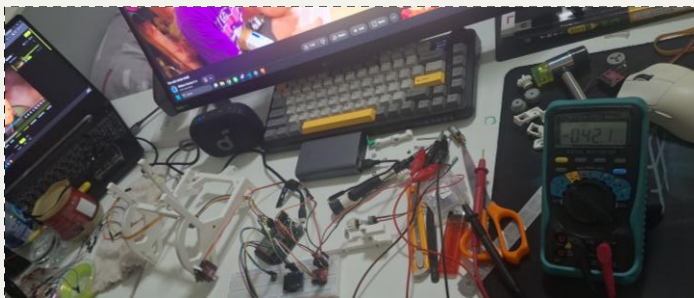
Sometimes launches
Sometimes doesn't.
50 / 50?

○ Status

Final Build & *Demo*

PICTURES

Drop your final-project photos in the slots below.



Problems & *Solutions*

PROBLEM

SOLUTION

01

I keep having problems with the latch design.

Change to Dog Clutch system.

02

String keeps snapping / string is too weak for holding rubber band's weight.

Change from using string to using nylon fishing line

03

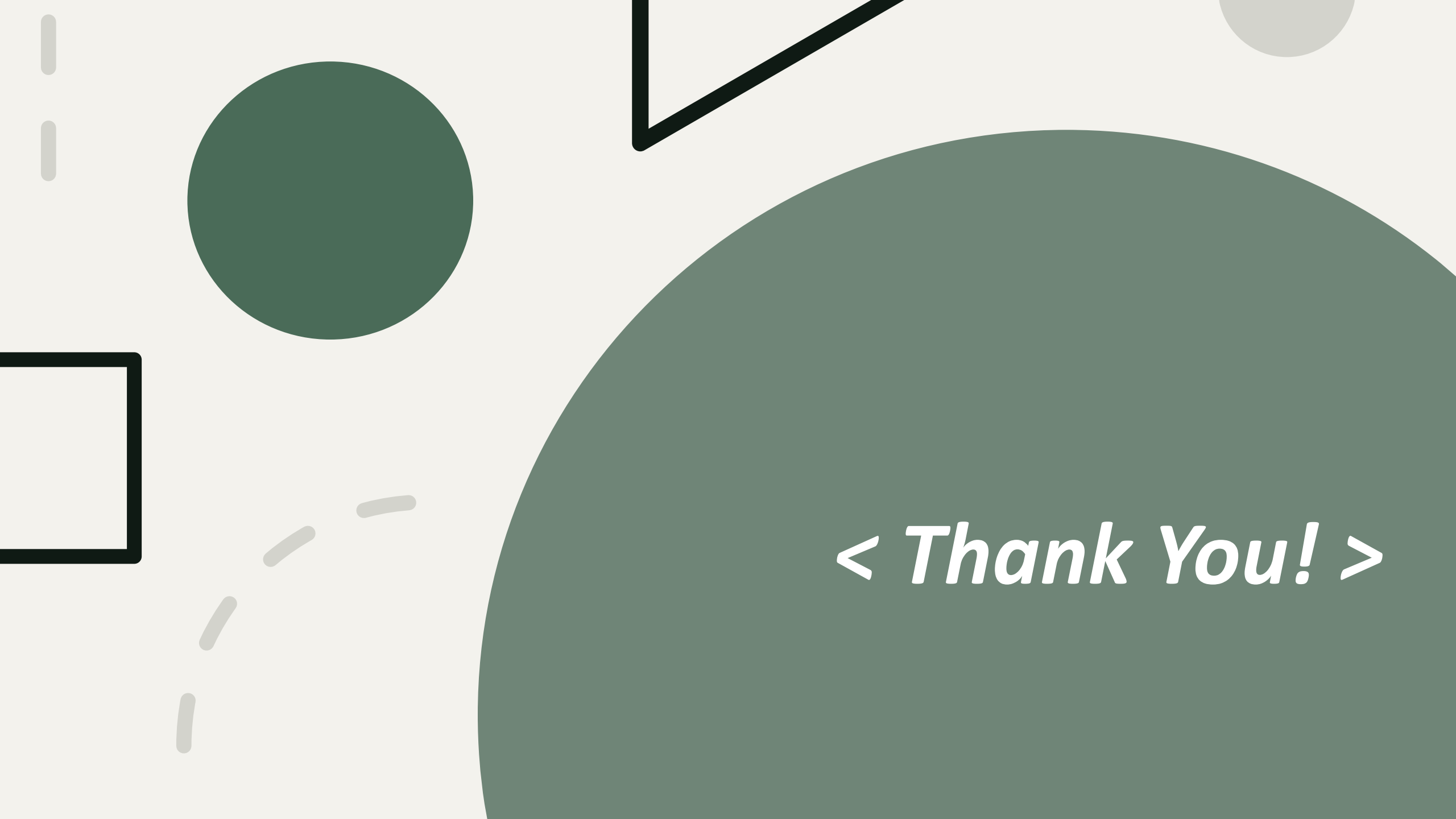
The MPU6050 (angle tracker) has a lot of noise and drifts a lot.

Combine and compare the gyro meter and accelerometer readings from the MPU6050 and filter it for more accurate and stable readings.

04

The arm overshoot or undershot the target angle.

Slow the motor in steps when getting close to target angle, then fix the rest with 45 ms nudges.



< Thank You! >